ECSE 551 Assignment 1: Lasso and ElasticNet Regression, SVM and MLP Classifiers, and S&P 500 Return Prediction

William Charron-Boyle, Radman Jamalshirpanjeh, Brandon Ku, Jack McDonald
Department of Electrical and Computer Engineering, McGill University, Montréal, QC, Canada

***Abstract*—In this assignment we explored two benchmark datasets, built four models from scratch and applied simple models to stock return prediction. Using only NumPy and gradient descent, we implemented Lasso regression with soft thresholding, ElasticNet regression, a one-vs-rest multiclass SVM and a one-hidden-layer MLP. We compared them with scikit-learn's `LinearRegression` and `RandomForestClassifier`. On the Diabetes dataset, our Lasso achieved the lowest test MSE (2797, versus 2822 for least squares). On the Digits dataset, the MLP was the most accurate classifier (98.0%), ahead of the random forest (97.2%) and the SVM (96.7%). We then predicted next-day volatility-adjusted S&P 500 returns with Lasso, random forest and MLP regressors, retrained every year on a rolling window. All models gave a positive Sharpe ratio on the 2011–2013 test period, ranging from 1.7 to 4.0. Adding sentiment data improved the results only slightly.**

I. Introduction

This assignment has three tasks. Task 1 explores two scikit-learn datasets. The Diabetes dataset [3] contains 442 patients and ten baseline variables: age, sex, body mass index (`bmi`), blood pressure (`bp`) and six serum measurements (`s1`–`s6`). Its target is a continuous measure of disease progression after one year. The Digits dataset [4] contains 1797 handwritten digits stored as 8×8 images, giving 64 pixel features with a label from 0 to 9.

Task 2 asks us to implement two regression models (ElasticNet and Lasso) and two classifiers (a multiclass SVM and an MLP) using only NumPy [2] and gradient descent. We compare them with `LinearRegression` [5] and `RandomForestClassifier` [6]. Every model uses a 70/30 train/test split with random state 42, and hyperparameters are chosen by 5-fold cross-validation (CV) on the training set.

Task 3 uses daily returns of S&P 500 stocks from 1980 to 2013, plus sentiment data for 2009–2015. The goal is to predict each stock's volatility-adjusted return for tomorrow, using only information available today, which we do with Lasso, random forest and MLP regressors from scikit-learn. We evaluate predictions with the Sharpe ratio, the profit-and-loss (PnL) curve, the cumulative rank correlation and the average holding period.

In short, regularisation helps only a little on Diabetes, but it makes the coefficients easier to interpret. On Digits, the non-linear models beat the linear SVM by a small margin. On the stock data, simple reversal and low-volatility signals give a weak but consistent edge out of sample. However, the best models change their positions almost every day.

II. Datasets

A. Diabetes and Digits

Neither dataset needed cleaning: there are no missing, infinite or duplicated values, and every value lies in a valid range. The Diabetes target ranges from 25 to 346, and the Digits pixels are integers from 0 to 16. Scikit-learn already centres and scales the Diabetes features (every feature has mean 0 and standard deviation ≈ 0.048), while the raw units vary widely, for example 19–79 for age. We still standardised both datasets using statistics from the training split only.

Fig. 1 shows that all Diabetes features are unimodal except `sex`, which is binary. The features `bmi`, `s3` and `s4` are right-skewed, `s4` is nearly discrete, and none has heavy tails. Box plots (in the notebook) show only a few outliers, and they are plausible values, so we kept them. Scatter plots of each feature against the target show roughly linear but noisy relationships, and `s1` against `s2` is almost a straight line. In the correlation matrix (Fig. 2), `bmi` ($r = 0.59$) and `s5` ($r = 0.57$) are the features most correlated with the target. Two pairs of features are strongly correlated with each other: `s1`–`s2` ($r = 0.90$) and `s3`–`s4` ($r = -0.74$). This collinearity is a good reason to try regularisation.

The ten Digits classes are balanced (174–183 images each), so accuracy is a fair metric. Almost half of all pixel values are zero. Pixels on the edges of the image are nearly always zero, while central pixels tend to be either very dark or very light (Fig. 3). Since the pixel variances differ a lot, we standardised the pixels. Most pixel pairs are only weakly correlated: 90.5% of the pairs have $|r| < 0.3$.

B. S&P 500 returns and sentiment

The return file covers 8577 trading days and 1335 stocks. The membership file marks between 489 and 500 index members on each day, and no member has a missing return. We only make and evaluate predictions for stocks that are members on day t. The Psychsignal sentiment data start in July 2009, but they cover few members: on average, 57 members per day in 2009, rising to 233 in 2013.

III. Methodology

The equations in this section follow the course lecture notes [1]. All from-scratch models are trained with gradient descent, $\theta_{t+1} = \theta_t - \eta\, \mathbf{g}_t$, where $\mathbf{g}_t$ is the gradient (or subgradient) of the loss and η is the learning rate. Hyperparameters

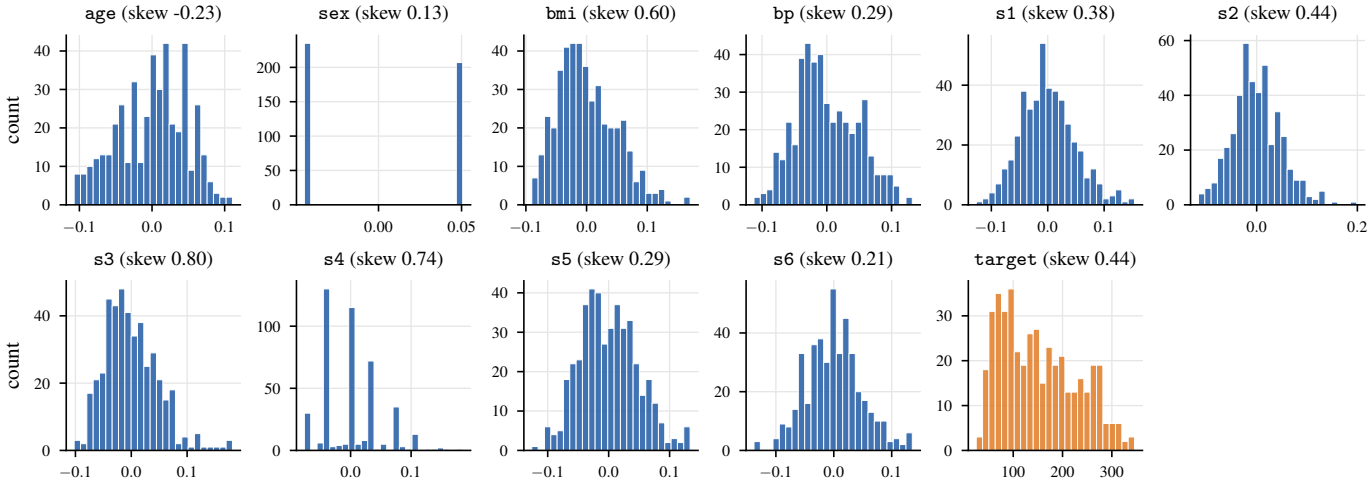


Fig. 1. Distributions of the Diabetes features and target. `sex` is binary, `s4` is nearly discrete, and the other features are unimodal and mostly slightly right-skewed.

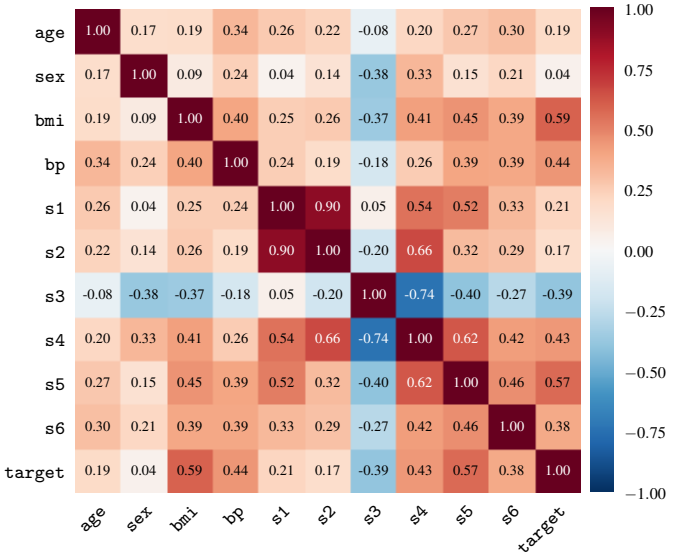


Fig. 2. Pearson correlation matrix of the Diabetes data. Note the strongly correlated pairs `s1`–`s2` and `s3`–`s4`.

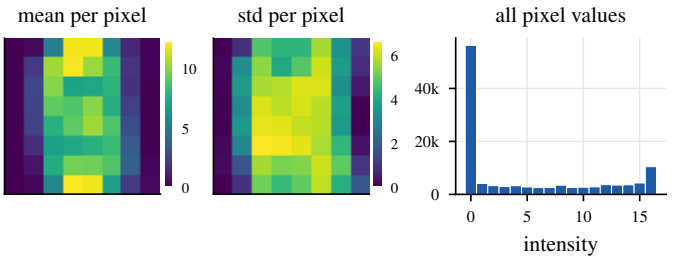


Fig. 3. Digits: mean and standard deviation of each pixel, and the histogram of all pixel values.

are chosen by 5-fold CV, which estimates the risk of a setting λ as $R^{\text{cv}}_\lambda = \frac{1}{K}\sum_{k=1}^{K} R_0\big(\hat\theta_\lambda(\mathscr{D}_{-k}), \mathscr{D}_k\big)$ with $K = 5$ [1]. In words, each model is trained on four folds and tested on the remaining one, and the five scores are averaged. We used the same folds (`KFold` [8]) for every model.

A. *Regression models*

Let $\mathbf{X}$ be the $m \times n$ standardised feature matrix, $\mathbf{y}$ the targets, $\mathbf{w}$ the weights and b the bias, which is never penalised. We report the mean squared error, $\text{MSE} = \frac{1}{m}\sum_i (y_i - \hat y_i)^2$, and the coefficient of determination, $R^2 = 1 - \sum_i (y_i - \hat y_i)^2 / \sum_i (y_i - \bar y)^2$, which is the fraction of the target variance explained by the model.

Linear regression. `LinearRegression` computes the ordinary least squares (OLS) solution $\hat{\mathbf{w}} = (\mathbf{X}^\top\mathbf{X})^{-1}\mathbf{X}^\top\mathbf{y}$ [1], with no penalty. It serves as our baseline.

Lasso. Lasso adds an ℓ_1 penalty to the squared error, which pushes some weights to exactly zero:

$$\min_{\mathbf{w},b} \; \frac{1}{2m}\|\mathbf{y} - (\mathbf{X}\mathbf{w} + b\mathbf{1})\|_2^2 + \lambda\|\mathbf{w}\|_1. \tag{1}$$

Since $|w_j|$ has no derivative at zero, we used the proximal gradient method from Lecture 8. Each iteration first takes a normal gradient step on the squared error,

$$\mathbf{v} = \mathbf{w} + \frac{\eta}{m}\mathbf{X}^\top\big(\mathbf{y} - \mathbf{X}\mathbf{w} - b\mathbf{1}\big), \tag{2}$$

and then applies soft thresholding to each weight:

$$w_j \leftarrow S_{\eta\lambda}(v_j) = \operatorname{sign}(v_j)\max\big(|v_j| - \eta\lambda, 0\big). \tag{3}$$

Weights smaller than $\eta\lambda$ in magnitude become exactly zero, and all other weights move towards zero by $\eta\lambda$.

ElasticNet. ElasticNet uses both an ℓ_1 and an ℓ_2 penalty:

$$\min_{\mathbf{w},b} \; \frac{1}{2m}\|\mathbf{y} - (\mathbf{X}\mathbf{w} + b\mathbf{1})\|_2^2 + \lambda_1\|\mathbf{w}\|_1 + \frac{\lambda_2}{2}\|\mathbf{w}\|_2^2. \tag{4}$$

The ℓ_2 term encourages correlated features to share similar weights, whereas Lasso tends to keep only one of them [1]. We used a single penalty, $\lambda = \lambda_1 = \lambda_2$, and plain subgradient descent with $\operatorname{sign}(w_j)$ as the subgradient of $|w_j|$:

$$\mathbf{w} \leftarrow \mathbf{w} - \eta\Big(-\tfrac{1}{m}\mathbf{X}^\top\mathbf{r} + \lambda\,\operatorname{sign}(\mathbf{w}) + \lambda\mathbf{w}\Big), \tag{5}$$

where $\mathbf{r} = \mathbf{y} - \mathbf{X}\mathbf{w} - b\mathbf{1}$ is the residual.

Learning rate and tuning. Gradient descent becomes unstable if η is too large compared with the largest eigenvalue L of $\mathbf{X}^\top\mathbf{X}/m$ (plus λ for ElasticNet). On our training data $L = 3.96$, so we used $\eta = 0.25$ for Lasso and $\eta = \min(0.25, 1/(3.96 + \lambda))$ for ElasticNet, with 5000 iterations each. We then chose λ from 25 values between 10^{-2} and 10^{2} using the CV MSE.

B. Classification models

Multiclass SVM. A binary SVM uses the decision function $f(\mathbf{x}) = \mathbf{w}^\top\mathbf{x} + w_0$ with labels $\tilde{y}_i \in \{-1, +1\}$. As shown in Lecture 8, the soft-margin SVM is equivalent to minimising the hinge loss with an ℓ_2 penalty:

$$\min_{\mathbf{w}, w_0} \; \frac{1}{2}\|\mathbf{w}\|_2^2 + \gamma \sum_{i=1}^{m} \max\big(0,\, 1 - \tilde{y}_i(\mathbf{w}^\top\mathbf{x}_i + w_0)\big). \tag{6}$$

Following the class template, we wrote the penalty as $\lambda\|\mathbf{w}\|^2$ and averaged the hinge losses over the samples. This is the same problem, rescaled, with $\gamma = 1/(2\lambda m)$. Only the samples inside the margin, $S = \{i : \tilde{y}_i f(\mathbf{x}_i) < 1\}$, contribute to the subgradient, which gives the updates

$$\mathbf{w} \leftarrow \mathbf{w} - \eta\Big(2\lambda\mathbf{w} - \frac{1}{m}\sum_{i \in S} \tilde{y}_i \mathbf{x}_i\Big), \quad w_0 \leftarrow w_0 + \frac{\eta}{m}\sum_{i \in S} \tilde{y}_i. \tag{7}$$

For ten classes, we trained one classifier per digit (that digit against all the others) and predicted the digit with the highest score [1]. We searched η from 0.001 to 3 and λ from 10^{-4} to 1, and then tuned the number of iterations.

MLP. Each layer of an MLP computes $\mathbf{z}_\ell = \varphi(\mathbf{b}_\ell + \mathbf{W}_\ell \mathbf{z}_{\ell-1})$ [1]. Our network has one hidden layer of H units with the activation $\mathrm{ReLU}(a) = \max(a, 0)$, followed by a softmax output, $\mu_c = e^{a_c} / \sum_{c'} e^{a_{c'}}$, which gives one probability per digit. We trained it with full-batch gradient descent on the cross-entropy loss from Lecture 6:

$$\mathrm{NLL}(\theta) = -\frac{1}{m}\sum_{i=1}^{m}\sum_{c=1}^{10} y_{ic} \log \mu_{ic}, \tag{8}$$

where y_{ic} is the one-hot label (from `OneHotEncoder` [9]). The gradients are computed by backpropagation, i.e. the chain rule applied layer by layer. At the output, the gradient is simply $(\mu - \mathbf{y})/m$, and it is passed back through the ReLU, whose derivative is 1 for positive inputs and 0 otherwise. We searched H from 8 to 512 and η from 0.01 to 2, and then tuned the number of epochs.

Random forest. A random forest averages many decision trees. Each tree is trained on a bootstrap sample of the data and considers a random subset of features at each split, which reduces variance [6]. We tuned the number of trees ($\{50, 100, 300\}$), the maximum depth ($\{\text{none}, 10, 20\}$), the number of features per split ($\sqrt{n}$ or $\log_2 n$) and the minimum leaf size ($\{1, 2\}$) with `GridSearchCV` [7], using the same five folds. The classifiers are compared using accuracy, i.e. the fraction of test images whose predicted digit matches the true label, and using confusion matrices, which show which digits get mixed up.

TABLE I
AVERAGE DAILY RANK CORRELATION OF EACH FEATURE WITH THE NEXT-DAY RETURN

	rev1	rev5	mom20	mom60	mom250	vol
1981–2000	0.001	−0.029	−0.018	−0.011	0.012	−0.016
2001–2010	−0.021	−0.014	−0.009	−0.005	0.011	−0.011

C. Return prediction

Problem formulation. We treat Task 3 as a cross-sectional regression problem. On each trading day t, for every stock that is an S&P 500 member on that day (about 500 stocks, well above the required 100), we predict the stock's volatility-adjusted return for day $t+1$ using only information available at the close of day t. Only the relative ordering of the stocks matters for a market-neutral portfolio, so the target is demeaned across stocks every day. We use the return data from 1981 to 2010 for training, with 2001–2010 serving as a validation period for choosing hyperparameters, and we keep 2011–2013 as the test set, as required by the assignment. The models are judged with the Sharpe ratio of the resulting portfolio, its PnL curve, the cumulative rank correlation and the average holding period.

Volatility adjustment. As described in the assignment, we track each stock's mean and variance with two exponentially weighted moving averages (EWMA): $\hat{\mu}_t = (1-\beta_2) r_t + \beta_2 \hat{\mu}_{t-1}$ and $\hat{\sigma}_t^2 = (1-\beta_1)(r_t - \hat{\mu}_t)^2 + \beta_1 \hat{\sigma}_{t-1}^2$, with $\beta_1 = 0.97$ and $\beta_2 = 0.99$. We drop the first 60 days of each stock as a burn-in period, while the estimates settle. The adjusted return is $\tilde{r}_t = r_t / \hat{\sigma}_{t-1}$.

Exploratory analysis. Volatility adjustment brings the standard deviation of member returns from 0.024 to about 1.06, and the adjusted series look roughly stationary. We built six candidate features on day t: the last return (`rev1`), the average return over 5, 20 and 60 days (`rev5`, `mom20`, `mom60`), a longer-term average that skips the most recent month (`mom250`), and the log volatility (`vol`). Before modelling, we measured each feature's average daily Spearman correlation with the next-day return (Table I). Every signal is very weak on its own ($|\rho| \le 0.03$), so the data have a very low signal-to-noise ratio. The signs show short-term reversal (recent winners tend to fall back), long-term momentum and a preference for low-volatility stocks. The one-day reversal `rev1` was close to zero before 2000 but clearly negative afterwards, which shows that these relationships change over time.

Data processing and leakage control. The adjusted returns are clipped to ± 5 so that a few extreme days do not dominate training. On each day, every feature is standardised across the member stocks (a cross-sectional z-score) and clipped to ± 3. We prevented look-ahead in four ways: (i) $\tilde{r}_t$ uses the volatility from day $t-1$; (ii) every feature on day t uses only returns up to day t; (iii) a model used in year Y is trained only on data from before year Y, and we drop the last training day so that no training target overlaps the prediction period; and (iv) all hyperparameters are chosen on 2001–2010, so the test years are never used for tuning.

Models and tuning. We compared three simple predic-

tors from scikit-learn: `Lasso` [10], which minimises the same objective as (1); `RandomForestRegressor` [6]; and `MLPRegressor` [11]. Each model is retrained every January on the previous W years (a rolling window). For Lasso, we searched $W \in \{5,10,20\}$ years and $\alpha \in \{10^{-4}, 10^{-3}, 3\cdot 10^{-3}, 10^{-2}\}$. The random forest uses 50 trees, considers half of the features at each split, and was tuned over (maximum depth, minimum leaf size) $\in \{(4,500),(8,200),(12,100)\}$. The MLP uses an ℓ_2 penalty of 10^{-3} and early stopping, and we tried hidden layers of (8), (32) and (32, 16). To keep training fast, the random forest and the MLP are trained on 200 000 randomly sampled rows from each window. Finally, we can smooth each stock's predictions over time with an EWMA of half-life $h \in \{0,1,2,5,10,20\}$ days.

Design rationale. Because the signal is weak and noisy, overfitting is the main risk, so every choice favours simple and stable models. Lasso's ℓ_1 penalty can remove features that do not help, although with the selected small penalty it kept all six. The random forest uses large minimum leaf sizes and limited depth, and the MLP is small, penalised and stopped early. The rolling window lets the models adapt as relationships such as the `rev1` reversal change over time, while ten years still give enough data to average out the noise. Demeaning the target removes the market's overall direction, so the models must actually separate good stocks from bad ones instead of profiting from the market's positive drift. Finally, smoothing is a cheap way to lengthen holding periods, since fast-changing predictions would be expensive to trade.

Metrics. On each day, the predictions $\hat{r}_{t,i}$ are demeaned across stocks, making the portfolio market neutral in volatility-adjusted terms, and scaled so that $\sum_i |\hat{r}_{t,i}| = 1$. The daily PnL and the annualised Sharpe ratio are

$$R_t = \sum_i \tilde{r}_{t+1,i}\,\hat{r}_{t,i}, \qquad \mathrm{SR} = \sqrt{252}\,\frac{\mathbb{E}[R_t]}{\sqrt{\mathrm{var}[R_t]}}, \tag{9}$$

with the benchmark set to zero and 252 trading days per year. We chose the Sharpe ratio because it measures return per unit of risk, which is what matters for a trading strategy and is much easier to interpret than an MSE on noisy returns. The cumulative PnL curve shows whether performance is steady over time or comes from a few good periods. We use the Spearman (rank) correlation because it checks whether the predicted ordering of stocks is right, and it is less sensitive to extreme returns than Pearson correlation. The average holding period,

$$H_{av} = \frac{1}{N}\sum_{i=1}^{N} \frac{\sum_t |\hat{r}_{t,i}|}{\sum_t |\hat{r}_{t,i} - \hat{r}_{t-1,i}|}, \tag{10}$$

where N is the number of stocks, estimates how many days a position is kept, which is a proxy for trading costs. Finally, the correlation of R_t with an equal-weighted long-only portfolio of the members checks how much the strategy simply follows the market.

TABLE II
DIABETES REGRESSION RESULTS ON THE TEST SET

Model	λ	Train MSE	Test MSE	R^2
LinearRegression (sklearn)	–	2924.0	2821.8	0.477
Lasso (ours)	0.681	2948.1	**2796.6**	**0.482**
ElasticNet (ours)	0.068	2953.6	2809.3	0.480

IV. RESULTS

A. Regression on Diabetes

With $\lambda = 1$, Lasso stayed stable up to $\eta = 0.5$ and ElasticNet up to about $\eta = 0.4$, which matches the limits $2/L$ we expected. Fig. 4 shows how the weights change as λ grows. Lasso sets weights to exactly zero one by one: `s2` is the first to stay at zero, and `bmi` is the last, at $\lambda \approx 68$. The last three weights to vanish are `bp`, `s5` and `bmi`, the features most correlated with the target. The earlier order does not follow the marginal correlations (`sex`, the least correlated feature, survives until $\lambda = 10$), because Lasso keeps a feature for what it adds given the other features, not for its correlation alone. ElasticNet shrinks the weights more smoothly, but they never reach exactly zero. This is a side effect of the subgradient update: the weights jump back and forth around zero instead of settling on it.

Cross-validation selected $\lambda = 0.681$ for Lasso and $\lambda = 0.068$ for ElasticNet. Both improved the CV MSE by only about 1% compared with no penalty. Table II shows that both of our models beat `LinearRegression` on the test set by a small margin, and all three explain about 48% of the variance. The gains are small because OLS barely overfits here: its training and test errors are close. The weights differ more than the errors. OLS gives large weights of opposite sign to the correlated pair `s1` (-43.3) and `s2` ($+24.3$). Lasso removes `s2` completely and reduces `s1` to -9.8, which makes the model easier to interpret at no cost in accuracy.

B. Classification on Digits

SVM. Fig. 5(a)–(b) shows that the SVM reaches about 94–95.5% CV accuracy over a wide range of settings. It fails only when both η and λ are large, because the updates then become unstable. The best setting was $\eta = 1$, $\lambda = 0.003$ and 500 iterations, which gave 96.7% test accuracy. The loss converged within about 100 iterations.

Fig. 6 shows the decision boundary of each one-vs-rest classifier on the test set. For classifier c, the score $f_c(\mathbf{x}) = \mathbf{w}_c^\top \mathbf{x} + b_c$ of every test image is plotted, and the boundary $f_c(\mathbf{x}) = 0$ is drawn in red. Digits 0, 2, 4, 6 and 7 are almost perfectly separated from the rest, with at most two mistakes each. Digits 1, 3, 8 and 9 overlap the boundary the most, and digits 8 and 9 have 11 misclassified test images each. Many images of these digits fall on the wrong side of their own boundary, but the final multiclass prediction still gets most of them right, because it picks the class with the highest score rather than requiring a positive one.

To also see the boundaries in the input space, we projected the data onto its first two principal components and retrained the SVM in 2-D (Fig. 7). The boundaries are straight lines, as

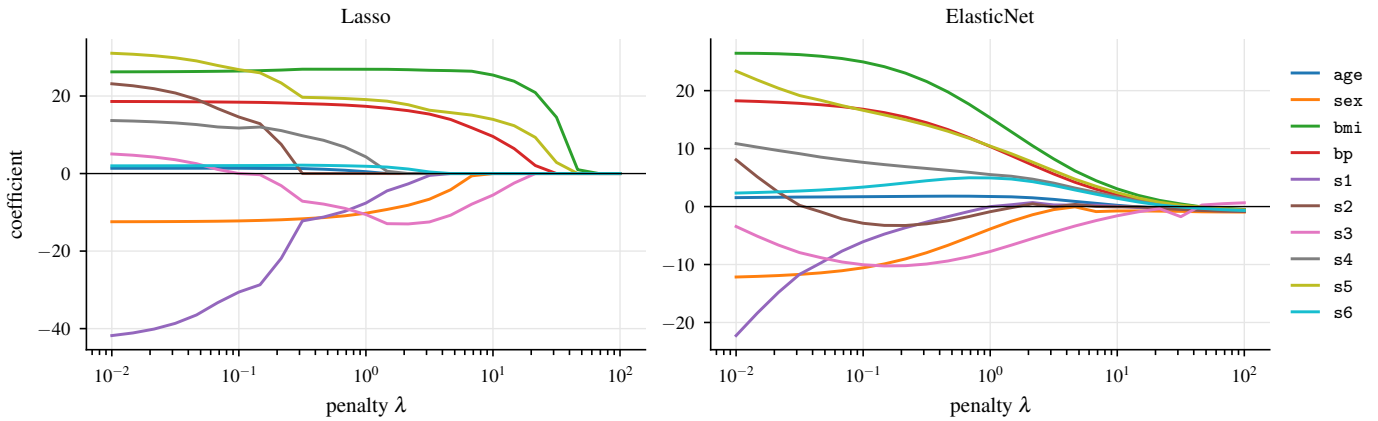


Fig. 4. Coefficient paths for Lasso (left) and ElasticNet (right) as the penalty λ increases. Lasso produces exact zeros, while ElasticNet shrinks the weights smoothly.

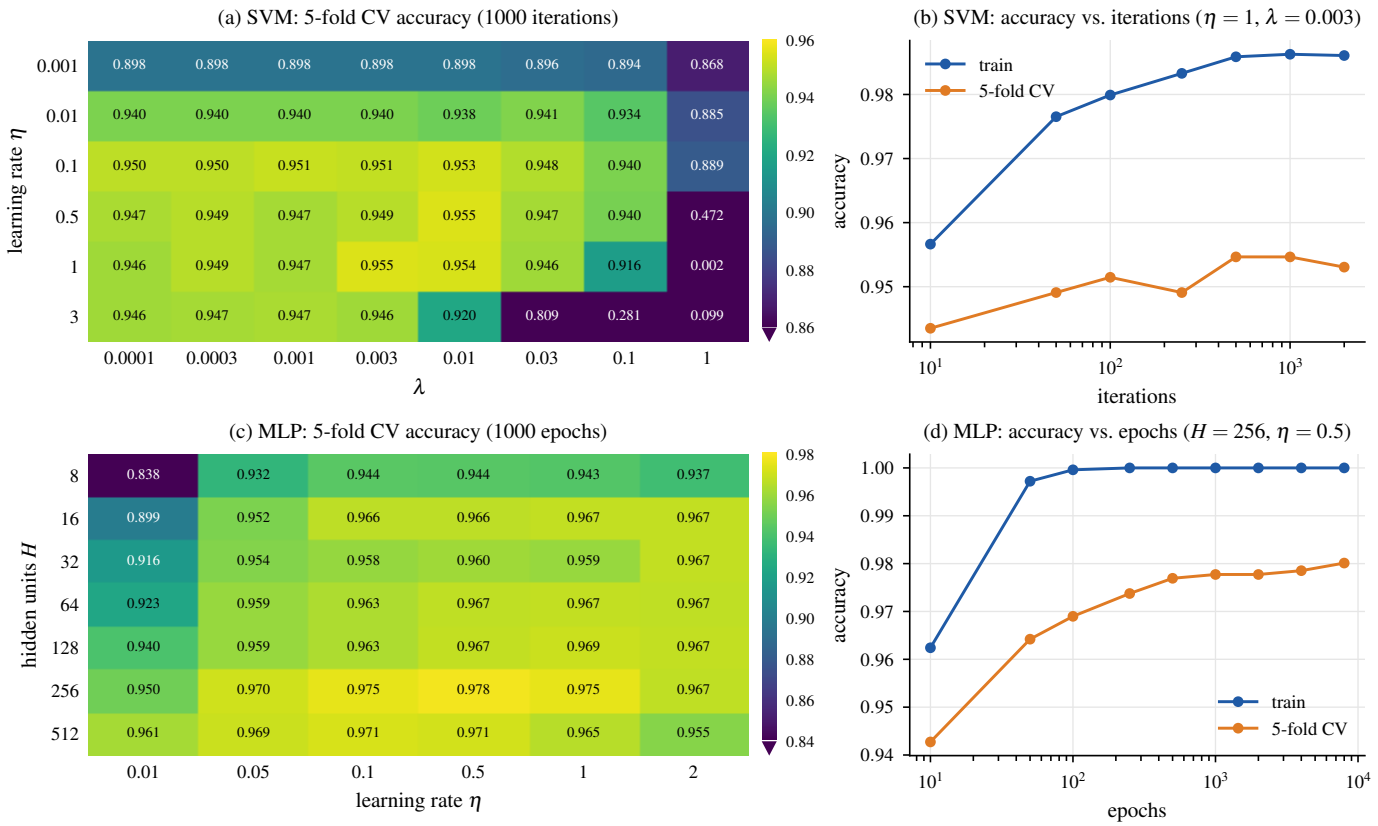


Fig. 5. Hyperparameter tuning of our classifiers on Digits. (a) SVM 5-fold CV accuracy for each learning rate and λ; cells below 0.86 are diverged runs. (b) SVM training and CV accuracy against the number of iterations. (c) MLP 5-fold CV accuracy for each hidden size and learning rate. (d) MLP training and CV accuracy against the number of epochs.

expected for a linear model. Two components keep only 22% of the variance, so the 2-D accuracy drops to 40–45%. Digits 0, 2, 4 and 6 still have clear regions, while the others overlap in the middle. A larger λ shrinks the weights, so some small class regions disappear and the accuracy drops slightly.

MLP. Fig. 5(c)–(d) shows that adding hidden units helps up to 256 units. A very small learning rate learns too slowly, and a very large one starts to hurt accuracy. The best setting was $H = 256$ and $\eta = 0.5$. Training accuracy reaches 100% after about 100 epochs, but the CV accuracy keeps rising slowly up to 8000 epochs, so longer training did not cause overfitting. We therefore trained the final MLP for 8000 epochs, and it reached 98.0% test accuracy.

Comparison. The best random forest used 300 trees with no depth limit, $\sqrt{n}$ features per split and a minimum leaf size of 1. Table III compares the three classifiers. The MLP makes 11 errors on 540 test images, the random forest 15 and the SVM 18, and the ranking is the same under CV. The

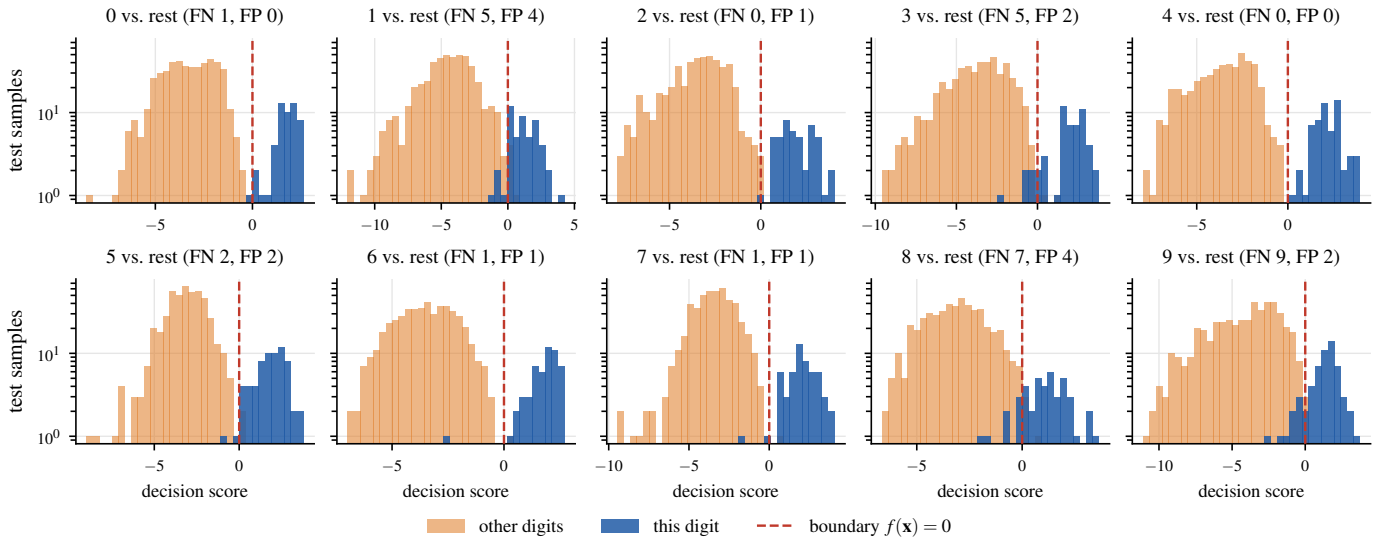


Fig. 6. SVM decision boundaries for the ten one-vs-rest classifiers. Each panel shows the test-set decision scores of the target digit (blue) and of all other digits (orange) on a log scale, with the boundary $f(\mathbf{x}) = 0$ in red. FN: images of the digit scored below 0; FP: other digits scored above 0.

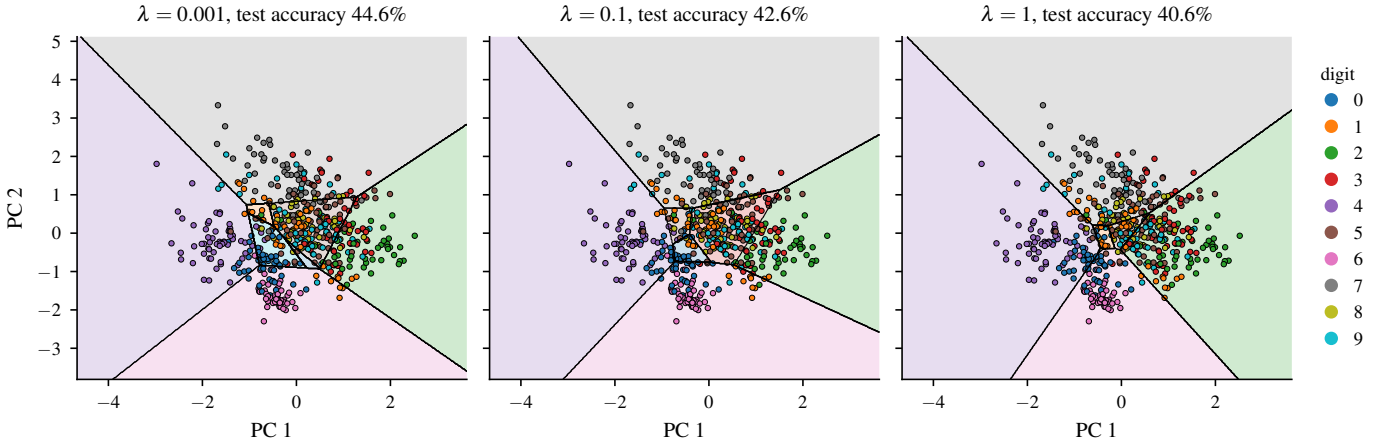


Fig. 7. SVM decision regions in the first two principal components for $\lambda = 0.001$, 0.1 and 1. Points are test samples coloured by their true digit.

TABLE III
DIGITS CLASSIFICATION RESULTS ON THE TEST SET

Model	CV acc.	Train acc.	Test acc.	Errors
Multiclass SVM (ours)	0.9547	0.9825	0.9667	18
MLP (ours)	**0.9801**	1.0000	**0.9796**	**11**
RandomForest (sklearn)	0.9745	1.0000	0.9722	15

confusion matrices (Fig. 8) show that most mistakes involve similar-looking digits such as 1, 3, 8 and 9. The SVM can only draw linear boundaries, whereas the MLP and the random forest can capture non-linear patterns, which explains their small advantage.

C. Return prediction

Tuning (2001–2010). For Lasso, we tried windows of 5, 10 and 20 years and four values of the penalty α. A 10-year window with $\alpha = 10^{-4}$ was best (Sharpe ratio 1.51): five years seem too noisy, and twenty years include relationships that no longer hold. Larger values of α removed useful features. Since 10^{-4} is the smallest value in the grid, every feature kept a non-zero weight and Lasso behaved almost like least squares. The Lasso weights were very stable from year to year. For the random forest, we tried three depths and chose depth 8 (Sharpe ratio 2.68). The deeper version scored slightly higher, but it changed positions faster than once per day and was more likely to overfit. For the MLP, one hidden layer of 32 units worked best (Sharpe ratio 2.43). Both non-linear models beat Lasso, but they held positions for only about one day.

Smoothing. Fig. 9 shows the trade-off between the Sharpe ratio and the holding period. Smoothing makes positions last longer but lowers the Sharpe ratio, especially for the random forest and the MLP. Lasso is less affected. We chose $h = 2$ as a compromise.

Test results (2011–2013). Table IV shows the metrics of (9) and (10) for each strategy. All six strategies had a positive Sharpe ratio on the test years and predicted at least 494 stocks

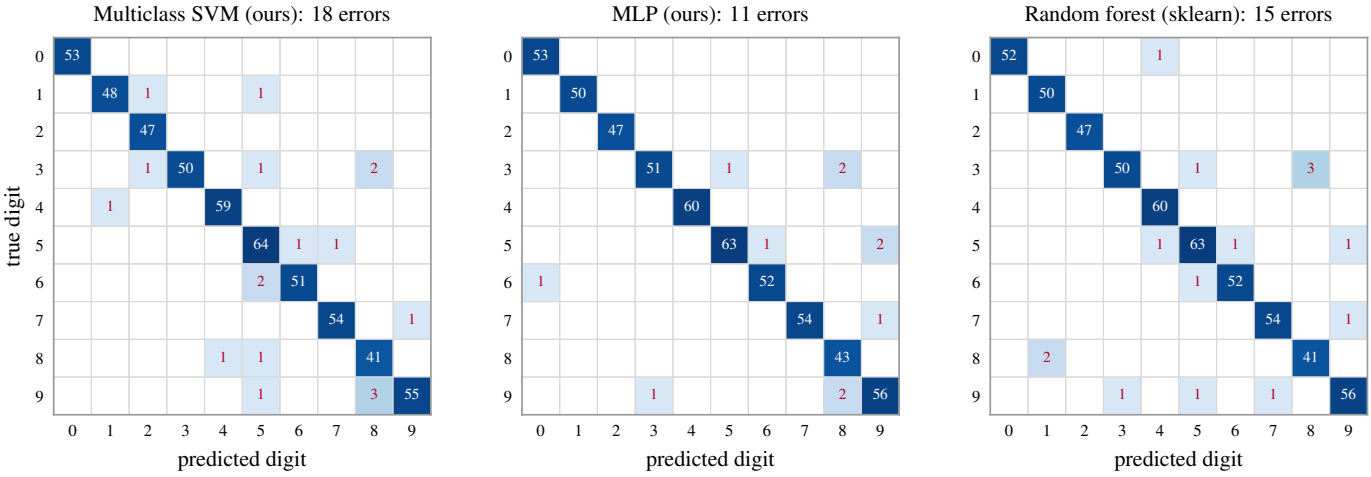


Fig. 8. Test-set confusion matrices for the SVM, the MLP and the random forest. Diagonal cells are correct predictions; empty cells are zero.

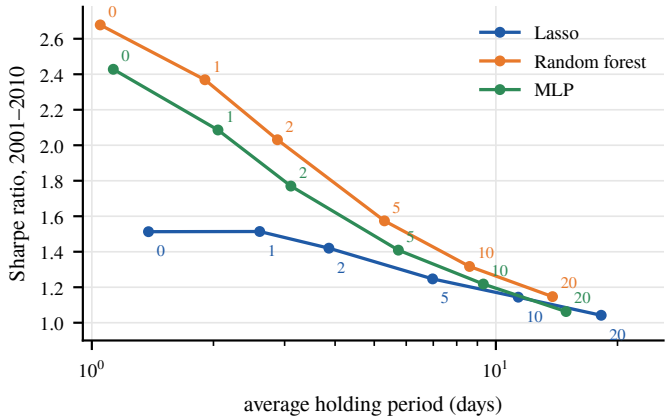


Fig. 9. Validation Sharpe ratio against holding period for different smoothing half-lives h (labels, in days).

TABLE IV
TEST RESULTS, 2011–2013

Strategy	Sharpe	Rank corr.	Hold. (days)	Mkt corr.
Lasso, $h = 0$	1.90	0.023	1.31	0.32
Lasso, $h = 2$	1.96	0.022	**4.26**	0.32
RF, $h = 0$	**4.03**	0.020	0.90	0.35
RF, $h = 2$	2.89	0.018	2.29	0.42
MLP, $h = 0$	2.47	0.016	1.06	**0.17**
MLP, $h = 2$	1.71	0.016	2.94	0.19

TABLE V
SHARPE RATIO BY SUB-PERIOD (FOR $h = 2$, SMOOTHING RUNS CONTINUOUSLY FROM 2001, SO THE 2011–13 VALUES DIFFER SLIGHTLY FROM TABLE IV)

Strategy	2001–05	2006–10	2011–13	2001–13
Lasso, $h = 0$	1.83	1.18	1.90	1.59
Lasso, $h = 2$	1.83	0.98	1.98	1.54
RF, $h = 0$	2.89	2.45	4.03	2.90
RF, $h = 2$	2.44	1.57	2.90	2.20
MLP, $h = 0$	2.31	2.66	2.47	2.43
MLP, $h = 2$	1.88	1.67	1.77	1.77

every day. The unsmoothed random forest scored highest (4.03), but it held positions for less than a day. Smoothed Lasso offered the best balance, with a Sharpe ratio of 1.96 and a holding period of 4.3 days. The assignment suggests that a Sharpe ratio of 0.8–1 is already interesting at a 3–5 day holding period. With three years of test data, the standard error of the Sharpe ratio is about $1/\sqrt{3} \approx 0.58$, so even the weakest result (1.71) is clearly above zero. The correlation with the market (0.17–0.42) is not zero. The portfolio is neutral in volatility-adjusted terms, but the low-volatility feature favours calm stocks, which receive larger dollar positions, so the dollar exposure is slightly net long.

Fig. 10 shows that the PnL and the cumulative rank correlation rise steadily over 2001–2013, with no long losing periods, and the test years look much like the validation years. Table V shows that 2006–2010 was the weakest sub-period for every model. Every model had a positive Sharpe ratio in each of the 13 years, with the weakest years during the 2007–2008 financial crisis.

Sentiment. We added two features from the Psychsignal data: net sentiment (bullish minus bearish) and activity (bullish plus bearish), each averaged over a week, with stocks that have no messages set to 0 (neutral). Because these data start in mid-2009, we retrained Lasso on up to three years of data, with and without them. Because of this shorter window, the baseline differs from Table IV. Sentiment raised the test Sharpe ratio from 1.78 to 1.92, or from 1.52 to 1.61 with smoothing. Given the limited coverage and the standard error of about 0.58, we do not consider this gain meaningful.

V. DISCUSSION AND CONCLUSION

On Diabetes, our Lasso and ElasticNet slightly beat least squares (test MSE 2797 and 2809, versus 2822). Their main benefit is simpler and more interpretable weights when features are correlated. Our ElasticNet never set weights exactly to zero. Using the proximal ElasticNet update from Lecture 8, $\mathbf{w} \leftarrow S_{\eta\lambda_1}(\mathbf{v})/(1+\eta\lambda_2)$, would fix this. On Digits, the MLP

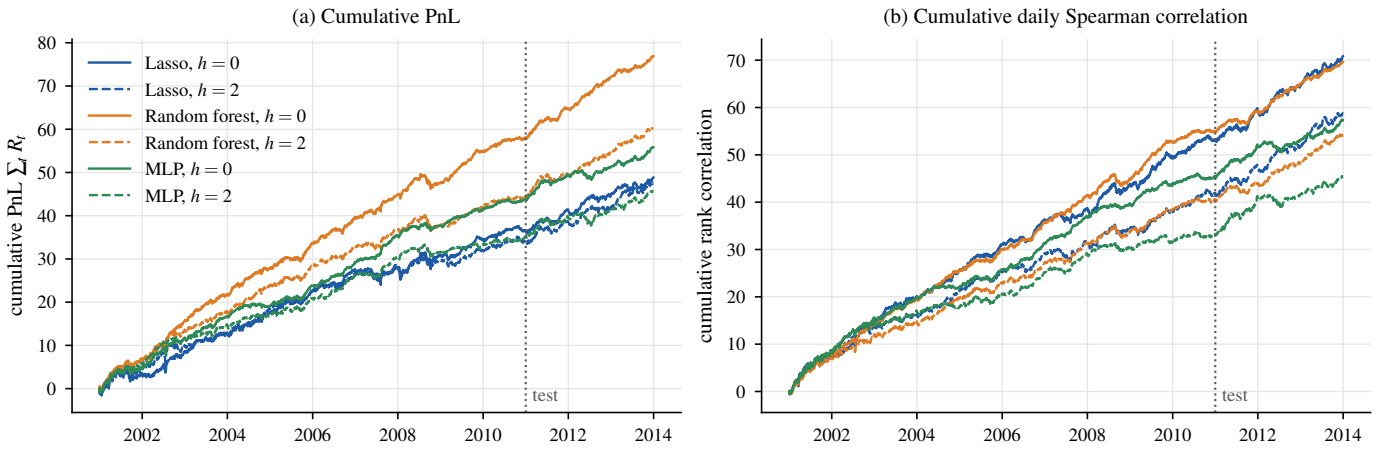


Fig. 10. Cumulative PnL (a) and cumulative daily rank correlation (b), 2001–2013. The dotted line marks the start of the test period. Solid lines: no smoothing; dashed lines: $h = 2$.

(98.0%) beat the random forest (97.2%) and the linear SVM (96.7%). Some non-linearity helps, although the linear model already performs well.

For stock returns, simple reversal, momentum and volatility features gave positive and fairly steady Sharpe ratios out of sample. The random forest and the MLP scored highest but traded almost daily, while Lasso was more stable and worked best once smoothed. The main limitations are that we ignored trading costs, which would hurt the fast strategies most, and that our grids for the non-linear models were small. In future work, we would add trading costs, control turnover more directly, remove sector effects and test more features while guarding against false discoveries.

VI. Statement of Contributions

Jack McDonald and William Charron-Boyle completed Tasks 1 and 2: the data analysis, the four from-scratch models and the comparison with scikit-learn. Radman Jamalshirpanjeh and Brandon Ku completed Task 3: the volatility adjustment, the features, the rolling training and evaluation, and the sentiment experiment. Each pair reviewed its own part of the notebook and wrote the corresponding sections of this report.

Use of large language models. We used LLMs in a limited, supporting role. Before writing any Task 3 code, we asked LLMs to explain the task's expectations and equations, using the prompt: "Explain Task 3 of this assignment step by step so I understand the expectations. For each equation, explain what it computes and why it is used." During coding, LLMs were also used to check the notebook against the assignment instructions and to revise a few notebook cells (the 2001–2013 Sharpe and PnL plots and some markdown summaries). For the report, they produced the initial IEEE template, formatted the tables and figures, and helped draft and edit the wording. All models, experiments and results come from our own notebook, and we checked every number in the report.

References

[1] M. Coates, *ECSE 551: Machine Learning for Engineers, Lecture Notes* (Lectures 3, 4, 6, 7, 8 and 9), McGill University, Fall 2026.

[2] NumPy Developers, "NumPy documentation," [Online]. Available: https://numpy.org/doc/stable/

[3] scikit-learn Developers, "load_diabetes," scikit-learn documentation. [Online]. Available: https://scikit-learn.org/stable/modules/generated/sklearn.datasets.load_diabetes.html

[4] scikit-learn Developers, "load_digits," scikit-learn documentation. [Online]. Available: https://scikit-learn.org/stable/modules/generated/sklearn.datasets.load_digits.html

[5] scikit-learn Developers, "LinearRegression," scikit-learn documentation. [Online]. Available: https://scikit-learn.org/stable/modules/generated/sklearn.linear_model.LinearRegression.html

[6] scikit-learn Developers, "Ensembles: Random forests," scikit-learn User Guide. [Online]. Available: https://scikit-learn.org/stable/modules/ensemble.html

[7] scikit-learn Developers, "GridSearchCV," scikit-learn documentation. [Online]. Available: https://scikit-learn.org/stable/modules/generated/sklearn.model_selection.GridSearchCV.html

[8] scikit-learn Developers, "KFold," scikit-learn documentation. [Online]. Available: https://scikit-learn.org/stable/modules/generated/sklearn.model_selection.KFold.html

[9] scikit-learn Developers, "Preprocessing data," scikit-learn User Guide. [Online]. Available: https://scikit-learn.org/stable/modules/preprocessing.html

[10] scikit-learn Developers, "Lasso," scikit-learn documentation. [Online]. Available: https://scikit-learn.org/stable/modules/generated/sklearn.linear_model.Lasso.html

[11] scikit-learn Developers, "MLPRegressor," scikit-learn documentation. [Online]. Available: https://scikit-learn.org/stable/modules/generated/sklearn.neural_network.MLPRegressor.html